

Sorting Algorithms

ITCS 4114: Real World Algorithms

Kalpathi Subramanian

Computer Science, UNC Charlotte

Last Modified: January, 2026

Learning Outcomes

- ♣ Explain the quadratic sorting algorithms (insertion, bubble and selection sort)
- ♣ Distinguish performance (based on input characteristics) performance of the 3 (insert, bubblesort, selection) quadratic sorting algorithms
- ♣ Explain **brute force** approach used to solve problems
- ♣ Explain **divide and conquer** approach used to solve problems
- ♣ Explain **decrease and conquer** approach used to solve problems
- ♣ Give examples of divide and conquer algorithms
- ♣ Give examples of decrease and conquer algorithms
- ♣ Explain the role of quadratic sorting algorithms in quicksort, mergesort algorithms
- ♣ Compare and contrast the quadratic sorting algorithms and the divide and conquer based sorting algorithms (quicksort, merge sort).
- ♣ Demonstrate understanding of quicksort, merge sort and the selection problems by working through a given example integer sequence.

Sorting Algorithms

Quadratic Sorting Algorithms

- ◇ Selection, Bubble and Insertion sort.
- ◇ Follows **Brute Force and Decrease and Conquer** designs.
- ◇ $O(N^2)$ algorithms, **simple**, but **not efficient for standalone use**, except for very small datasets
- ◇ Good **complement** to the more efficient algorithms

Efficient Sorting Algorithms

- ◇ Merge sort, Quicksort
- ◇ Follow a **Divide and Conquer** based algorithmic strategy
- ◇ Both have $O(N \log N)$ performance, but merge sort is guaranteed $\Theta(N \log N)$ for worst case, while Quicksort is $O(N^2)$.
- ◇ Overhead of Quicksort is smaller than merge sort; with careful design, quicksort **can avoid the worst case**.

Selection Sort

Approach

- ◇ **Brute force** design
- ◇ Search the array for the smallest element, exchanges with the first element, find the second smallest with the second element..
- ◇ *Animation Link*

Algorithm

```
// Input: Array A[0.. , n-1]
// Output: Sorted Array A[0..n-1]
for (i = 0 to (n-2) {
    min = i
    for (j = (i+1) to (n-1) {
        if (A[j] < A[min])
            min = j
    }
    swap(A[i] , A[min])
}
```

Analysis: Selection Sort

- ◇ Count up the operations in the two loops.

$$\begin{aligned}T(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 \\&= \sum_{i=0}^{n-2} (n-1) - (i+1) + 1 \\&= \sum_{i=0}^{n-2} (n-1-i) \\&= (n-1) + (n-1-1) + \dots + (n-1) - (n-2) \\&= (n-1) + (n-2) + \dots + 1 \\&= \sum_{i=1}^{n-1} 1 = (n-1)n/2\end{aligned}$$

Thus, $T(n) \in \Theta(n^2)$

Bubble Sort

Approach

- ◇ Also **Brute force** design
- ◇ Scan the array and **bubble** the largest element to the end of the array, **repeat until no more swaps occur**
- ◇ Good for **nearly sorted arrays**
- ◇ *Animation Link*

Algorithm

```
// Input: Array A[0.., n-1]
// Output: Sorted Array A[0..n-1]
Boolean swap_flag = true;
while (swap_flag){
    swap_flag = false
    for (i = 1 to (n-1) {
        if (A[i-1] > A[i]){
            swap(A[i-1], A[i])
            swap_flag = true;
        }
    }
}
```

Analysis: Bubble Sort

- ◇ More complicated, as performance can range from **linear** to quadratic in the size of the input.
- ◇ Linear time when?
- ◇ Quadratic time when?
- ◇ **Worst case analysis** similar to selection sort analysis

$$\begin{aligned} T(n) &= \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 \\ &= (n-1) * n/2 \end{aligned}$$

Thus, $T(n) \in O(n^2)$

Insertion Sort

An example of a **Decrease and Conquer** algorithms

- ◇ Array has a sorted section and an unsorted section
- ◇ Element from the unsorted section is inserted into its correct position in the sorted section by moving the elements in front of it by 1 position
- ◇ *Animation Link*

Algorithm

```
InsertionSort (A[0...n-1]) {  
    for (i = 1 to n-1) { // A[0...i] is sorted  
        v = A[i]  
        j = i-1  
        while (j ≥ 0 and (A[j] > v) {  
            A[j+1] = A[j]  
            j—  
        }  
        A[j+1] = v  
    }  
}
```


Insertion Sort: Analysis

- Algorithm complexity is somewhat dependent on the input
- What is the best case?
- Worst case happens, if the **inner loop runs through the entire array to find the insertion point**
- Average case** efficiency is **half** as much as the worst case efficiency and makes it preferable to bubble or selection sorts

$$\begin{aligned}T(n) &= \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1 \\&= \sum_{i=1}^{n-1} i \\&= (n-1)n/2\end{aligned}$$

Thus $T(n) \in O(n^2)$

Final Notes: The Quadratic Sorting Algorithms

Recap

- ◇ Insertion sort is the preferred among the 3 algorithms for its performance, reduces (like bubble sort) to $\Theta(n)$ for nearly sorted lists.
- ◇ For data that is nearly sorted, bubble sort is the preferred algorithm (reduces to $\Theta(n)$)
- ◇ **Strength:** These algorithms are simple and good for use on small lists, and often used by Quicksort, Mergesort when the lists become small, or as a final pass when the input is nearly sorted.

Questions

- ◇ What is the worst case input data for bubble sort? Why?
- ◇ What is the worst case input data for insertion sort? Why?
- ◇ Does selection sort performance depend on input data set?

Questions/Problems: Quadratic Sorting Algorithms

- ◇ Best case input data for bubble sort? Worst case Input?
- ◇ Best case input for insertion sort? Worst case Input?
- ◇ Why is bubble sort efficient on nearly sorted lists?
- ◇ Is there a best or worst case input for selection sort?
- ◇ Modify each algorithm for sorting values in descending order.

Divide and Conquer Design

What is Divide and Conquer

- ◇ **Divide** the problem into a number of sub-problems
- ◇ **Conquer** the sub-problems by solving them **recursively**
- ◇ **Combine** the sub-problem solutions to solve the original problem.
- ◇ **Example Divide and Conquer Algorithms:** Merge sort, Quicksort
- ◇ **Analysis:** Can use the **Master Theorem**

Note:

- ◇ Not every problem using Divide and Conquer is more efficient than brute force. For example the sum of n numbers

$$S(n) = a_0 + a_1 + \cdots + a_{n-1}$$

Analyzing Divide and Conquer Algorithms

- ◇ Divide conquer algorithms divide the problem into b instances, each of size n/b , with a of them needing to be solved, resulting in a recurrence given by

$$T(n) = aT(n/b) + f(n)$$

where $T(n/b)$ represents the cost of solving the sub problems and $f(n)$ is the cost of combining the sub-problem solutions.

- ◇ Many such recurrences can be solved by the **Master Theorem** under certain conditions.

Master Theorem

If $f(n) \in \theta(n^d)$, $d \geq 0$, in the above recurrence, then

$$T(n) \in \begin{cases} \theta(n^d) & \text{if } a < b^d \\ \theta(n^d \log n) & \text{if } a = b^d \\ \theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Master Theorem: Examples

Summing the first n numbers; assume division into 2 equal sized problems

$$S(n) = (a_0 + a_1 + \cdots + a_{\lfloor n/2-1 \rfloor}) + (a_{\lfloor n/2 \rfloor} + \cdots + a_{n-1})$$

Assume division into 2 equal sized problems, so

$$\begin{aligned} T(n) &= aT(n/b) + f(n) \\ &= 2T(n/2) + 1 \end{aligned}$$

($f(n) = O(1)$, since combining the 2 solutions is an addition, which is constant time)

Applying the Master Theorem (3rd case will apply here),

$$a = 2, b = 2, d = 0, \text{ so } a = 2, b^d = 2^0 = 1 \text{ and } a > b^d$$

$$\text{Thus, } T(n) \in \theta(n^{\log_b a}) = \theta(n^{\log_2 2}) = \theta(n)\sqrt{}$$

Sample Problems: Master Theorem

If $f(n) \in \theta(n^d)$, $d \geq 0$, in the above recurrence, then

$$T(n) \in \begin{cases} \theta(n^d) & \text{if } a < b^d \\ \theta(n^d \log n) & \text{if } a = b^d \\ \theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

Use the Master theorem to solve the following problems:

- ◇ $T(n) = 4T(n/2) + n$
- ◇ $T(n) = 16T(n/4) + n$
- ◇ $T(n) = 4T(n/2) + n^2$
- ◇ $T(n) = 4T(n/2) + n^3$
- ◇ $T(n) = 2T(n/4) + n^{0.51}$
- ◇ $T(n) = 4T(n/2) + cn$
- ◇ $T(n) = 3T(n/2) + n^2$
- ◇ $T(n) = 3T(n/3) + \sqrt{n}$
- ◇ $T(n) = 7T(n/3) + n^2$

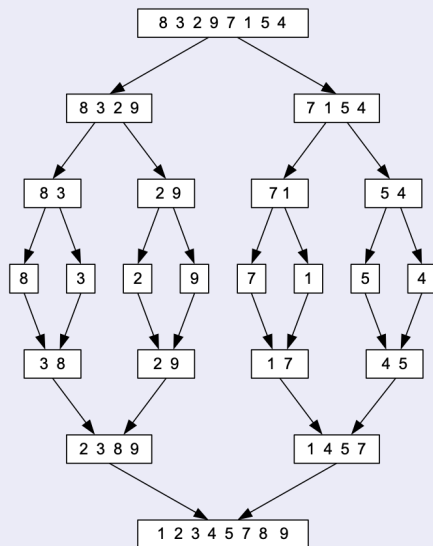
Merge Sort

An efficient $\theta(N \lg N)$ sorting algorithm

- ◇ **Divide:** Split array $A[0, \dots, N - 1]$ in two equal halves, make copies of each half, B and C
- ◇ **Conquer:** Sort arrays B and C recursively
- ◇ **Combine:** Merge sorted arrays B and C into the array A as follows:
- ◇ Repeat until no elements remain in **one** of the arrays:
 - ▷ Compare the first elements in the remaining unprocessed portions of the two arrays
 - ▷ Copy the smaller of the two into A; increment the associated array index
 - ▷ If all elements in one of the arrays are processed, append the remaining unprocessed elements from the other array into A.

Merge Sort

Example



Algorithm

```
MergeSort (A[0...N-1]) {  
    if (n > 1) {  
        copy A[0...( $\lfloor n/2 \rfloor - 1$ )] to B[0...( $\lfloor n/2 \rfloor - 1$ )]  
        copy A[( $\lfloor n/2 \rfloor$ )...(n-1)] to C[0...( $\lceil n/2 \rceil - 1$ )]  
        MergeSort (B[0...( $\lfloor n/2 \rfloor - 1$ )])  
        MergeSort (C[0...( $\lceil n/2 \rceil - 1$ )])  
        Merge(B, C, A)  
    }  
}  
Merge(B[0..p-1], C[0..q-1], A[0..p+q-1]) {  
    while (i < p and j < q) { // merging the sub arrays  
        if (B[i] <= C[j]) {  
            A[k] = B[i];  
            i++;  
        }  
        else {  
            A[k] = C[j];  
            j++;  
        }  
        k++;  
    }  
    if (i == p) // check for remaining unprocessed elements  
        copy C[j..q-1] into A[k..p+q-1];  
    else copy B[i..p-1] into A[k..p+q-1];  
}
```

Merge Sort

Complexity Analysis

- ◇ All cases (best, worst, average) efficiency: $\theta(n \lg n)$
- ◇ Comparisons in mergesort follow the recurrence:

$$T(n) = 2T(n/2) + T_{\text{merge}}(n), \text{ for } n > 1, T(1) = 0$$

- ◇ Worst case for T_{merge} is $n - 1$, so

$$T_{\text{worst}}(n) = 2T(n/2) + n - 1$$

i.e. $f(n) = n$

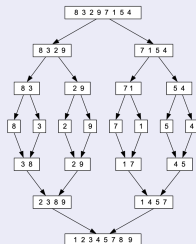
- ◇ Using the Master theorem,

$$a = 2, b = 2, d = 1 (n^1 = n), \text{ so } a = b^d$$

$$\text{Thus, } T(n) = \theta(n^d \lg n) = \theta(n^1 \lg n) = \theta(n \lg n) \checkmark$$

Merge Sort

Complexity Analysis



- ◇ All of the work is done in the merging (Conquer) process.
- ◇ How much?
 - ▷ At each level of the merge process, worst case you have n comparisons
 - ▷ How many such levels are there? Think of a binary search tree!
 - ▷ Its the same as the height of the binary search tree, which $\log n$.
 - ▷ Hence total cost is $O(n \log n)$

Merge Sort

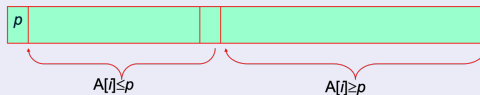
- ◇ Complexity is $\Theta(n \lg n)$, (best, worst and average cases)
- ◇ Total comparisons very close to theoretical minimum
- ◇ Overhead can be high, requires additional space for temporary arrays
- ◇ Array copying can be avoided (in place sorting), but very complex algorithm, of little practical use
- ◇ Can also sort bottom up (starting with pairs of values at a time) and merging - **avoids stack overhead**
- ◇ **Multiway merge sort** – sort into more than two parts – useful in sorting files in secondary memory

Quicksort

Overview

- ◇ Invented by C.A.R. Hoare
- ◇ Follows a **Divide and Conquer** algorithmic strategy
- ◇ An $N \lg N$ algorithm, but worst case can degrade to $O(N^2)$
- ◇ Careful choices can **avoid the worst case**
- ◇ **Less overhead (no array copies)** compared to mergesort

Idea: Partitioning based on a chosen pivot element



- ◇ Choose a pivot element (first, random, or median of 3)
- ◇ Partition the list based on the pivot (see above)
- ◇ Swap pivot with the last element of left partition.
- ◇ Recursively sort the two sublists.

Quick Sort

Algorithm

```
// Input: Subarray of  $A[0..n-1]$ , defined by indices  $l, r$ 
// Output: Subarray  $A[l..r]$  in sorted order
Quicksort ( $A[l..r]$ ) {
    if ( $l < r$ ) {
         $s = \text{Partition}(A[l..r])$  //  $s$  is the pivot index
        Quicksort( $A[l..s-1]$ )
        Quicksort( $A[s+1, r]$ )
    }
}

Partition( $A[l..r]$ ) {
    // Input: Subarray of  $A[0..n-1]$ , defined by indices  $l, r$ 
    // Output: Partition of  $A[l..r]$  with split position returned
     $p = A[l]$  // uses first position, for illustration
     $i = l$ ;  $j = r+1$ ;
    do {
        do  $i++$  while  $A[i] \leq p$ ;
        do  $j--$  while  $A[j] \geq p$ ;
        swap ( $A[i], A[j]$ )
    }
    while ( $i < j$ )
    swap ( $A[i], A[j]$ ) // undo last swap
    swap ( $A[l], A[j]$ ) // swap with pivot
    return  $j$  // final pivot position
}
```

Quick Sort

Partitioning Example

0	1	2	3	4	5	6	7	
5	3	1	<u>9</u>	8	2	<u>4</u>	7	
			<i>i</i>			<i>j</i>		
5	3	1	4	<u>8</u>	<u>2</u>	9	7	
			<i>i</i>	<i>j</i>				
5	3	1	4	2	8	9	7	
				<i>j</i>	<i>i</i>			// i crosses j
5	3	1	4	8	2	9	7	// last (incorrect) swap
5	3	1	4	2	8	9	7	// undo last swap
2	3	1	4	5	8	9	7	// swap with pivot
2	3	1	4					
	<i>i</i>	<i>j</i>						
2	1	3	4					
	<i>j</i>	<i>i</i>						// i crosses j
2	3	1	4					// incorrect swap
2	1	3	4					// undo swap
1	2	3	4					// swap with pivot

Quick Sort

Complexity Analysis

- ◇ The two loops in the partitioning traverse at most $n + 1$ elements.
- ◇ **Best Case:** All splits result in the equal sized partitions; then the recurrence is

$$C_{best}(n) = 2C_{best}(n/2) + n, n > 1, C_{best}(1) = 0$$

which, by Master Theorem is $\Theta(n \lg n)$.

- ◇ **Worst Case:** All partitions result in 1 partition being empty, the second element of size $(n-1)$, resulting in a complexity of $\Theta(n^2)$

$$C_{worst}(n) = \sum_{i=3}^{n+1} i = \frac{(n+1)(n+2)}{2} - 3 = \Theta(n^2)$$

Quick Sort

Complexity Analysis: Average Case: Must consider all possible sized partitions

- ◇ **Average Case:** Tricky proof
- ◇ In each case, you will perform $(n + 1)$ comparisons and then need to consider the size of the partitions
- ◇ For a partition at index s , the left partition has s elements and right partition has $n - 1 - s$ elements
- ◇ The recurrence is as follows:

$$C_{avg}(n) = \frac{1}{n} \sum_{s=0}^{n-1} [(n + 1) + C_{avg}(s) + C_{avg}(n - 1 - s)] \text{ for } n > 1,$$
$$C_{avg}(0) = 0, C_{avg}(1) = 0$$

Turns out

$$C_{avg}(n) \approx 2n \ln n \approx 1.39 n \log_2 n$$

Improving Quicksort

- ◇ Ideally you want the median value for the partitioning to create equal partitions; computing the median (QuickSelect method) is at least $O(n)$!
- ◇ Can avoid the worst case by using a **Median of 3 algorithm** to choose the pivot; pick the median of the first, last and middle elements of the partition
- ◇ For large datasets, when the sublists becomes small, switch to insertion sort; nearly sorted lists are sorted quickly by insertion sort.
- ◇ When the sublists become quite small, stop quicksort; then run insertion sort on the whole array - very fast for nearly sorted lists.

Quicksort Problems

- ◇ Apply the Hoare partition to the following values:

73, 6, 57, 88, 85, 42, 83, 72, 48, 60

Show partitioned values of the above set and then a partitioning of the left partition. Use the **first value** as pivot in each case.

- ◇ Apply quicksort to the following example:

E, X, A, M, P, L, E

using the first letter as the pivot

- ◇ Given an array of positive and negative numbers, design an algorithm to rearrange the values so that all the negative numbers precede the positive numbers. What is the alg. complexity?
- ◇ **Dutch National Flag Problem:** Given the characters *R*, *W*, *B*, colors of the Dutch national flag in an array, rearrange so that Rs come before Ws and Ws come before Bs.

Selection Problem

Given a set of orderable elements, find the k th smallest item

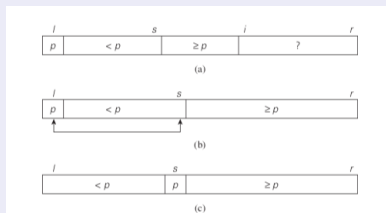
- ◇ **Application:** Find the median of a set of numbers.
- ◇ In quicksort, picking the median value as the pivot will result in $\Theta(N)$ performance, but determining the median is at least $O(N)$.
- ◇ Sorting the entire list is one way to solve the problem, but its overkill! We can do better.
- ◇ Similar to Quicksort, we will use a partitioning approach

Selection Problem

Lomuto Partitioning: Approach

- Similar to quicksort, choose a pivot element and partition the elements into two subsets:

	s	
all elements are $< A[s]$	$A[s]$	all elements are $\geq A[s]$



- Once all elements are processed, partition is complete and swap the pivot with the element at the split position.
- The split position element corresponds to the s 'th smallest value in the array

Quickselect

Algorithm

```
// Partitions subarray using first element as pivot
// Input: A subarray A[l..r]
// Output: Partition of A[l..r]
LomutoPartition(A[l..r]) {
    p = A[l]
    s = l
    for i = l + 1 to r {
        if A[i] < p)
            s = s + 1;
            swap(A[i], A[s])
    }
    swap(A[l], A[s]) // move pivot to split position
    return s
}

//Solves the selection problem
//Input: Subarray A[l..r] of array of orderable elements and
// integer k ( $1 \leq k \leq r - l + 1$ )
//Output: The value of the kth smallest element in A[l..r]
Quickselect(A[l..r], k) {
    s = LomutoPartition(A[l..r])
    if (s == (k-1))
        return A[s]
    else if (s > (l + k - 1))
        Quickselect(A[l, ..., s-1], k)
    else
        Quickselect(A[s+1, ..., r], k-1-s)
}
```

Complexity Analysis

- Running through the Lomuto partition of an n element array takes $n - 1$ operations. In the best case, you find the solution in 1 iteration,

$$C_{best} = n - 1 \text{ operations} = \Theta(n)$$

- Can have extremely unbalanced partitions, so worst case,

$$C_{worst} = (n - 1) + (n - 2) + \dots + 1 = (n - 1)n/2 = \Theta(n^2)$$

- Average case performance is typically **linear**
- More sophisticated algorithms focus on cleverly choosing the pivot and are of linear complexity in the worst case!

Example

0	1	2	3	4	5	6	7	8
<i>s</i>	<i>i</i>							
4	1	10	8	7	12	9	2	15
<i>s</i>	<i>i</i>							
4	1	10	8	7	12	9	2	15
<i>s</i>							<i>i</i>	
4	1	10	8	7	12	9	2	15
<i>s</i>							<i>i</i>	
4	1	2	8	7	12	9	10	15
<i>s</i>								<i>i</i>
4	1	2	8	7	12	9	10	15
2	1	4	8	7	12	9	10	15

0	1	2	3	4	5	6	7	8
<i>s</i>	<i>i</i>							
4	1	10	8	7	12	9	2	15
<i>s</i>	<i>i</i>							
4	1	10	8	7	12	9	2	15
<i>s</i>							<i>i</i>	
4	1	10	8	7	12	9	2	15
<i>s</i>							<i>i</i>	
4	1	2	8	7	12	9	10	15
<i>s</i>								<i>i</i>
4	1	2	8	7	12	9	10	15
2	1	4	8	7	12	9	10	15

Exercise

Given the sequence **9, 12, 5, 17, 20, 30, 8**, find its median using the Quickselect algorithm